

UNIDADE 1

INTRODUÇÃO E ERROS

Neste capítulo, vamos discutir alguns conceitos envolvidos na etapa de modelagem matemática, principalmente erros e suas medidas. Na Seção 1.1 vamos apresentar as etapas da solução de um problema, que envolvem a modelagem e a sua resolução. Na Seção 1.2, vamos apresentar o conceito de algoritmo, fundamental para o desenvolvimento de métodos numéricos. Na Seção 1.3 temática. Em seguida vamos discutir os tipos de erros envolvidos durante o processo de modelagem e solução de um problema via métodos numéricos. Na Seção 1.4 apresentamos de forma breve a aritmética de ponto flutuante e os erros de arredondamento e truncamento. Finalizamos a primeira parte da disciplina, com uma lista de exercícios envolvendo os conceitos estudados.

1.1 Modelagem matemática

Diversos problemas reais são descritos por modelos matemáticos. De forma bem resumida, um modelo matemático é composto por um conjunto de equações que descrevem (ou representam) um problema ou fenômeno físico. Para apresentar um modelo matemático para um problema, por exemplo o espalhamento de doenças ou previsão do tempo, precisamos discutir alguns conceitos fundamentais.

Vamos começar com as etapas de solução de um problema prático. De forma bem simplificada, vamos apresentar duas etapas descritas em [2] e ilustrada na Figura 1.1.

- **Modelagem Matemática:**

A primeira etapa envolve a Modelagem do problema, ou seja, nosso objetivo é obter um conjunto de equações e relações que represente da maneira mais conveniente o problema de interesse. Inicialmente é necessário equacionar o processo que pretendemos estudar,

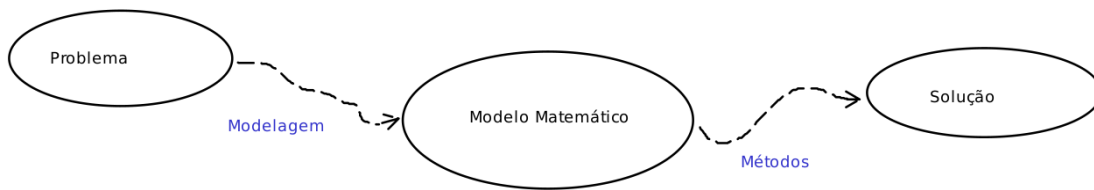


Fig. 1.1: Etapas da solução de um problema.

ou seja, decidir quais são as variáveis relevantes e como elas se relacionam, quais são os dados do problema, quais leis regem o fenômeno estudado, etc.

Por exemplo, para representar o movimento de um fluido, em geral são utilizadas como incógnitas (variáveis) as componentes da velocidade e a pressão, em uma região (domínio) considerada. As leis de conservação (massa, quantidade de movimento, energia, etc...), bem como, equações constitutivas são usadas para estabelecer a relação entre as variáveis escolhidas, dentre elas as incógnitas. Como, em geral, nem tudo que influi no fenômeno pode ser considerado, algumas simplificações (hipóteses simplificadoras, por exemplo, considerar alguma das variáveis, com a densidade constante) são consideradas neste estágio. Um outro exemplo, se queremos maximizar o lucro de uma empresa, precisamos definir a função objetivo, ou seja, uma expressão que envolva as variáveis associados aos lucros e aos custos da empresa. A função objetivo é a função a ser otimizada, ou seja, no fornecerá o máximo lucro. Além disso, a empresa possui restrições, por exemplo, pode produzir uma quantidade limitada de um determinado produto. As restrições devem ser satisfeitas e fazem parte do modelo matemático.

- **Solução do Problema:**

Uma vez estabelecido o Modelo Matemático (as equações), o próximo passo é resolver o problema. Nem sempre é possível obtermos soluções por meio de métodos analíticos, onde as soluções são apresentadas por meio de expressões. Por exemplo, para obtermos as raízes de uma equação do segundo grau, podemos utilizar a fórmula de Bhaskara. Entretanto, não temos uma metodologia analítica para se obter as raízes de uma função qualquer. Assim, no contexto da disciplina, vamos apresentar soluções aproximadas, obtidas por meio de métodos numéricos para diversos problemas.

De qualquer forma, nesta etapa podemos obter uma solução “ruim”, seja a metodologia analítica ou numérica. Neste caso, pode ser necessário repensarmos a forma de resolver o problema ou a etapa de modelagem.

Vamos utilizar métodos numéricos para resolver os problemas propostos. Os métodos numéricos podem ser representados por meio de algoritmos. Por isso, na próxima seção vamos apresentar o conceito de algoritmo e os conceitos fundamentais de programação.

1.2 Algoritmos e conceitos básicos de programação

Um algoritmo é um processo sistemático para a resolução de um problema. Em outras palavras, pode ser entendido como um procedimento que descreve, sem ambiguidade, uma sequência de passos a ser executada, para se atingir um objetivo. O desenvolvimento de algoritmos é importante para problemas a serem resolvidos em um computador. Um algoritmo computa uma *saída* (a resposta de um problema), a partir de uma *entrada* (informações/dados inicialmente conhecidos) que permitem determinar a solução do problema [6]. Durante o processo de computação o algoritmo manipula dados gerados a partir de sua entrada, por meio de instruções.

A estrutura básica de um programa (um algoritmo implementado) em diferentes linguagens de programação consiste em:

- **Entrada:** os dados de entrada são inicializados, recebidos por meio dos dispositivos de entrada ou de um arquivo.
- **Saída:** A resposta fornecida pelo algoritmo é transmitida para os arquivos de saída ou salva em um arquivo.
- **Lógica e aritmética:** operações aritméticas e lógicas que resolvem o problema de interesse.
- **Condiciona:** verifica se uma condição é satisfeita antes de executar uma sequência de instruções.
- **Repetição:** executa um bloco de instruções repetidamente.

Para mais detalhes veja bons cursos de Introdução à Programação disponíveis em [3] e [4].

Já estudamos diversos algoritmos desde que iniciamos nossos estudos na escola.

Exemplo 1 O Algoritmo 1 recebe dois números inteiros a e b , com $b \neq 0$ e devolve o quociente da divisão a/b .

Algoritmo 1: Divisão inteira

Recebe a e b

$c \leftarrow 0$

enquanto $a > b$ **faça**

$c \leftarrow c + 1$
 $a \leftarrow a - b$

Devolve c

Vamos simular o Algoritmo 1 para a entrada $a = 13$ e $b = 5$. Começamos com a inicialização da variável $c = 0$. Em seguida, verificamos a condição $a > b$. Neste caso, $13 > 5$ cujo resultado é **verdadeiro**, então atualizamos os valores de c e a , respectivamente, $c = 0 + 1 = 1$ e $a = 13 - 5 = 8$. Retornamos para verificar a condição $a > b$, pois estamos executando um comando de repetição. O valor da variável a foi atualizado, temos $8 > 5$

cujo resultado é **verdadeiro**. Assim, atualizamos os valores de c e a , respectivamente, $c = 1 + 1 = 2$ e $a = 8 - 5 = 3$. Novamente vamos verificar a condição $a > b$, sendo $a = 3$ e $b = 5$, cujo resultado é **falso**. Assim, o algoritmo termina e o valor do quociente da divisão está armazenado na variável c . Note que a variável a fica com o resto da divisão.

Os algoritmos são implementados por meio de uma linguagem de programação, utilizando tipos de dados e operações suportadas pelo computador. Neste texto, os algoritmos serão apresentados por meio de pseudocódigo, para que possam ser implementados na linguagem de sua preferência.

Na descrição dos algoritmos ao longo do texto, são utilizadas as seguintes convenções.

- A linguagem possui uma estrutura de blocos de instruções semelhante ao *Python*. O início e o final de cada bloco são determinados pela indentação. Por exemplo, o bloco iniciado por **enquanto** no Algoritmo 1 inclui as duas linhas seguintes.
- Variáveis simples e vetores armazenam dados. Os elementos de vetores são identificados por índices entre colchetes. Por exemplo, $S[2]$ indica o terceiro elemento do vetor S . Vamos utilizar a indexação começando em zero.
- A atribuição de valores é indicada pelo símbolo $\leftarrow$.
- As sentenças após o símbolo $\#$ indicarão comentários.
- As seguintes declarações são empregadas para representar:
 - Condicional:


```
se ... então
se ... então ... caso contrário
```
 - Repetição:


```
enquanto ... faça
para ... faça
```

Exemplo 2 O Algoritmo 2 recebe uma sequência de números armazenada em um vetor e faz a inversão da ordem dos termos desta sequência.

Algoritmo 2: Inversão de uma sequência

```
Recebe  $S$  de tamanho  $n + 1$ 
para  $i = 0, \dots, \lfloor n/2 \rfloor$  faça
     $\text{temp} \leftarrow S[i]$ 
     $S[i] \leftarrow S[n - i]$ 
     $S[n - i] \leftarrow \text{temp}$ 
```

O algoritmo altera o próprio vetor, então a saída do algoritmo está armazenada no próprio vetor (dado de entrada). A notação $\lfloor x \rfloor$ significa *piso* de x e representa o maior inteiro menor ou igual a x .

Vamos fazer a simulação do algoritmo com a entrada $S = [-1, 2, 0, 3, 7]$. O tamanho do vetor (número de elementos) é 5 e temos $S[0] = -1$, $S[1] = 2$, $S[2] = 0$, $S[3] = 3$ e

$S[4] = 7$. A repetição começa em $i = 0$ e vai até $i = \lfloor 4/2 \rfloor = 2$. Para $i = 0$ temos $\text{temp} = S[0] = -1$, $S[0] = S[4 - 0] = 7$ e $S[4] = \text{temp} = -1$. Assim, após a primeira iteração temos $S = [7, 2, 0, 3, -1]$. Para $i = 1$ temos $\text{temp} = S[1] = 2$, $S[1] = S[4 - 1] = 3$ e $S[3] = \text{temp} = 2$. Assim, após a segunda iteração temos $S = [7, 3, 0, 2, -1]$. Note que S já está invertido, mas mais uma iteração é feita. Para $i = 2$ temos $\text{temp} = S[2] = 0$, $S[2] = S[4 - 2] = 0$ e $S[2] = \text{temp} = 0$. Essa iteração é necessária, caso o tamanho do vetor seja um número par.

Exemplo 3 O Algoritmo 3 recebe um número natural n e calcula a soma dos n termos da Série Harmônica: $\sum_{k=1}^{\infty} \frac{1}{k}$.

Algoritmo 3: Soma de n termos da Série Harmônica

Recebe n # quantidade de termos

$S \leftarrow 0$

para $i = 1, \dots, n$ **faça**

$S \leftarrow S + \frac{1}{i}$

Devolve S

Antes de iniciar, vale ressaltar que estaremos usando o símbolo “.” (ponto) em vez de “,” (vírgula) na representação de números reais.

A Série Harmônica é uma série numérica, ou seja, uma soma infinita de termos de uma sequência de números. Vamos fazer a simulação do algoritmo com a entrada $n = 4$. A variável S é inicializada com zero pois irá armazenar a soma dos n primeiros termos da série. Para $i = 1$ temos $S = 0 + 1/1 = 1$. Para $i = 2$ temos $S = 1 + 1/2 = 1.5$. Para $i = 3$ temos $S = 1.5 + 1/3$. Aqui esbarramos no nosso primeiro problema numérico: Como vamos representar $1/3$? Ao fazermos qualquer tipo de aproximação, estamos incluindo um tipo de erro: o erro de representação do número. Para darmos continuidade na simulação, vamos representar $1/3$ com quatro algarismos, 0.3333 (note que zeros à esquerda não são considerados). Assim temos, $S = 1.5 + 0.3333 = 1.8333$. Para encerrar, com $i = 4$ temos $S = 1.8333 + 1/4 = 2.0833$. Assim, a soma aproximada dos quatro primeiros termos da Série Harmônica é 2.0833.

Como no exemplo anterior, a maioria dos processos de solução de um problema, desde a etapa de modelagem são inseridos erros/incertezas. O conjunto destas incertezas vai afetar o resultado final. É necessário termos conhecimento dos tipos de erros envolvidos e como eles se propagam. Além disso, vamos tratar dos tipos de erros envolvidos na representação dos números e em cálculos aritméticos.

1.3 Erros

A solução obtida por meio de um método numérico, em geral, difere da solução do problema real. Definimos **erro** como a diferença entre a solução exata e a solução aproximada de um problema.

Se v é uma aproximação do valor u , definimos o **erro absoluto**, e como:

$$e = |u - v|. \quad (1.1)$$

Já o **erro relativo**, e_r é dado por,

$$e_r = \frac{|u - v|}{|u|} \quad (1.2)$$

Agora, se $\mathbf{v}, \mathbf{u} \in \mathbb{R}^n$, a medida do erro $\mathbf{e} \in \mathbb{R}^n$ será dada por uma das normas, a norma-2,

$$\|\mathbf{e}\|_2 = \left(\sum_{i=1}^n e_i^2 \right)^{\frac{1}{2}}, \quad (1.3)$$

e a norma ∞ (ou norma do máximo), dada por,

$$\|\mathbf{e}\|_\infty = \max_{1 \leq i \leq n} (|e_i|). \quad (1.4)$$

A partir de agora, vamos descrever brevemente algumas fontes de erros.

- **Erros iniciais:**

Nos erros iniciais são incluídas as incertezas da modelagem matemática, com as hipóteses simplificadoras, as medições de parâmetros, as condições iniciais, etc. São erros que já existem antes mesmo de começarmos o processo de resolução.

A influência das perturbações iniciais dependem da estabilidade do problema. Em alguns casos, a influência deste tipo de erro pode inviabilizar a solução numérica, são problemas conhecidos como mal postos ou mal condicionados, problemas que são sensíveis a pequenas perturbações nos dados de entrada.

- **Erros de truncamento:**

Os erros de truncamento são erros associados a parada de um processo infinito. Por exemplo, para aproximar a exponencial de um número $x \in \mathbb{R}$. Para discutirmos este exemplo, vamos precisar relembrar o conceito da **Série de Taylor** de uma função.

A Série de Taylor de uma função $f(x)$ é a representação de $f(x)$ como a soma infinita de termos calculados a partir das derivadas de $f(x)$ em um ponto x_0 , dada por

$$f(x) = f(x_0) + \sum_{n=1}^{\infty} \frac{f^{(n)}(x_0)}{n!} (x - x_0)^n, \quad (1.5)$$

com $x, x_0 \in \mathbb{R}$, $f^{(n)}$ a n -ésima derivada de $f(x)$. Assim, se x_0 pertence a uma vizinhança de x , ao truncarmos a Série de Taylor, obtemos uma aproximação para $f(x)$. Dado x_0 ,

considere $x = x_0 + h$, ou seja, x está em uma vizinhança de x_0 , de tamanho h . Se truncarmos a Série, no termo K temos a seguinte aproximação,

$$f(x) = f(x_0) + \sum_{n=1}^K \frac{f^{(n)}(x_0)}{n!} (h)^n + O(h^K), \quad (1.6)$$

sendo $O(h^K)$ representa a ordem do erro cometido, ou seja, erro $\leq Ch^K$, com C uma constante e h um número pequeno, pois h representa uma vizinhança.

Exemplo 4 Considere $f(x) = e^x$, com $x = 0.5$. Aproxime o valor de $e^{0.5}$ usando cinco termos da Série de Taylor.

A Série de Taylor de e^x em torno de $x_0 = 0$ é dada por:

$$e^x = 1 + x + \frac{x^2}{2} + \frac{x^3}{3!} + \dots + \frac{x^n}{n!} + \dots = 1 + \sum_{k=1}^{\infty} \frac{x^k}{k!}$$

Note que para chegarmos na expressão acima, estamos considerando $x_0 = 0$ e então $x = h$. Além disso, qualquer derivada de e^x é e^x e quando avaliada em x_0 temos $f^{(n)}(x_0) = 1$.

Para conseguirmos, aproximar o valor de $e^{0.5}$ precisamos “truncar” a soma infinita. Utilizando cinco termos, temos:

$$e^{0.5} \approx 1 + 0.5 + \frac{0.5^2}{2} + \frac{0.5^3}{6} + \frac{0.5^4}{24}$$

Mais uma vez temos um problema com a representação de números reais. Quantos algarismos devemos utilizar para representar um número real? Como o erro na representação influencia na solução aproximada? Para este exemplo, vamos utilizar a precisão fornecida por uma calculadora científica: $e^{0.5} \approx 1.6484375$. Vamos considerar o valor fornecido pela avaliação da função exponencial em uma calculadora científica como o valor exato: $e^{0.5} = 1.6487212707$. Agora, vamos calcular o erro absoluto: erro = $|1.6484375 - 1.6487212707| = 0.0002837707$.

- **Erros de arredondamento:**

Este tipo de erro está associado a representação do sistema de numeração da máquina calculadora. Conforme visto nos exemplos anteriores, precisamos definir a regra de arredondamento na representação de números reais. Por exemplo, considere o número π , como vamos representá-lo? O primeiro passo é definir quantos algarismos serão usados para representá-lo e depois usar uma das regras de arredondamento. Vamos utilizar as seguintes regras de arredondamento:

1. **Corte:** desconsideramos os algarismos a partir da quantidade de algarismos fixada. Por exemplo, o número 1.6666666 representado com 4 algarismos é 1.666.

2. **Arredondamento:** Considere que fixamos n algarismos para representar um número. Se o algarismo $n + 1$ for maior ou igual a 5 somamos 1 ao algarismo n , caso contrário, usamos o corte. Por exemplo, usando o arredondamento, o número 1.666666 é representado com 4 algarismos por 1.667, pois o quinto algarismo é 6.

Para discutir os erros de arredondamento, primeiro vamos apresentar a representação de números inteiros e números reais em máquinas que possuem uma quantidade limitada de algarismos para representar estes números. Na sequência, vamos utilizar estes conceitos para realizar as operações aritméticas com uma quantidade de algarismos fixa.

1.4 Aritmética de ponto flutuante

Os números podem ser representados em diferentes sistemas de numeração. No sistema decimal, trabalhamos com a base 10 e os símbolos: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9. Por exemplo: $1234 = 1 \times 10^3 + 2 \times 10^2 + 3 \times 10^1 + 4 \times 10^0$. Já no sistema binário, utilizamos a base 2 e os símbolos: 0, 1. Por exemplo, o número 13 representado na base binária é: $1101 = 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$. Como exercício, faça sucessivas divisões pela base e analise o resto destas divisões.

Assim um número inteiro no sistema binário é representado pelos coeficientes de sua expansão binária,

$$N = a_n \times 2^n + a_{n-1} \times 2^{n-1} + \dots + a_1 \times 2^1 + a_0 \times 2^0 \Rightarrow N = (a_n a_{n-1} \dots a_1 a_0)_2.$$

Agora, vamos iniciar a análise da representação de números reais. Lembre-se que estaremos usando o símbolo “.” (ponto) em vez de “,” (vírgula) na representação de números reais. Na base decimal um número real pode ser representado da seguinte maneira:

$$A = a_n \times 10^n + a_{n-1} \times 10^{n-1} + \dots + a_1 \times 10^1 + a_0 \times 10^0 + a_{-1} \times 10^{-1} + \dots + a_{-n} \times 10^{-n} + \dots$$

Por exemplo, o número $13.311 = 1 \times 10^1 + 3 \times 10^0 + 3 \times 10^{-1} + 1 \times 10^{-2} + 1 \times 10^{-3}$.

Dada uma base $b \geq 2$ temos a representação dada por,

$$A = a_n \times b^n + a_{n-1} \times b^{n-1} + \dots + a_1 \times b^1 + a_0 \times b^0 + a_{-1} \times b^{-1} + \dots + a_{-n} \times b^{-n} + \dots, \quad (1.7)$$

com $0 \leq a_i < b$ para todo i . Por exemplo, o número 1001.101 na base $b = 2$ representa,

$$1 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3} = 9.625.$$

Agora vamos apresentar a **Aritmética de ponto Flutuante**. Ela é usada em cálculos científicos e é uma padronização na representação de números reais. Com ela podemos:

- (i) especificar como representar números reais com precisão simples e precisão dupla; e,
- (ii) padronizar o arredondamento nas operações aritméticas.

A representação em ponto flutuante de um número x , em uma base b , com n algarismos significativos (quantidade de algarismos na mantissa do número) é definida por,

$$fl(x) = \pm(0.d_1d_2 \dots d_n) \times b^e, \quad (1.8)$$

sendo $d_1d_2 \dots d_n$ a mantissa do número (representação fracionária) na base b , com $d_1 \neq 0$, e (número inteiro) o expoente do número, com $e_1 < e < e_2$, e_1 e e_2 , o menor e o maior expoente, respectivamente.

Exemplo 5 Note que, 0.2531×10^{-1} representa o número 0.02531 na base 10 com 4 algarismos significativos. Já o número 0.2531×10^2 , que possui a mesma mantissa do número anterior, representa 25.31 na base 10.

Para representar um sistema de numeração de ponto flutuante, vamos utilizar a seguinte notação: $F(b, n, e, E)$, onde b é a base, n é a quantidade de algarismos significativos. Note que, teremos um sistema limitado, pelo maior número representado e pelo menor número representado pelo sistema.

Exemplo 6 Dado o sistema $F(10, 5, -3, 4)$, o número $x = 279.15$ é representado neste sistema como $x = 0.27915 \times 10^3$.

Qual é o maior número representado neste sistema de numeração? Note que, 0.99999×10^4 representa a maior mantissa com o maior expoente. Uma máquina com este sistema de numeração não representa números maiores que 9999.9 (*overflow*).

Qual o menor número positivo representado neste sistema de numeração? Note que, 0.10000×10^{-3} representa a menor mantissa ($d_1 \neq 0$) com o menor expoente. Números positivos menores que 1×10^{-4} não serão representados neste sistema (*underflow*).

Agora, o que acontece se quisermos representar o número π no sistema apresentado no Exemplo 6?

Dada uma quantidade fixa de algarismos significativos não conseguimos representar todos os números reais em uma máquina.

Uma aproximação do número π será representada por $\pi \approx 3.1415$ usando a regra do corte e $\pi \approx 3.1416$ usando a regra de arredondamento, pois o sexto algarismo (lembre-se que o sistema do exemplo trabalha com cinco algarismos significativos) é igual a 9.

Exemplo 7 O número 0.11100110×2^2 na base 2 com 8 algarismos significativos representa o número 3.59375. Já o número 0.11100111×2^2 representa 3.609375.

Note que no Exemplo 7, o número 0.11100111 é o próximo número a ser representado na máquina, depois do número 0.11100110. Isso significa que os números reais entre 3.59375 e 3.609375 não são representados neste sistema de ponto flutuante.

Na padronização organizada pelo IEEE (*Institute for Electrical and Eletronics Engineers*) estabeleceu-se que:

- Um número real com precisão simples é representado por 32 *bits* (4 *bytes* pois 1 *byte* - 8 *bits*), sendo: 1 *bit* reservado para o sinal, 8 *bits* reservados para o expoente e 23 *bits* reservados para a mantissa.
- Um número real com precisão dupla é representado por 64 *bits*, sendo 1 *bit* para o sinal, 11 *bits* para o expoente e 52 *bits* para a mantissa.

Neste caso, na base 2, um número real com precisão dupla, é representado por:

$$(-1)^s 2^{e-1023} (1 + m),$$

sendo s o bit do sinal, e o expoente e m a mantissa.

- o expoente e com 11 *bits* fornece uma variação de 0 a $2^{11} - 1 = 2047$, que é normalizada de -1023 a 1024 .
- Os 52 *bits* na mantissa equivalem a aproximadamente 16 algarismos (na base decimal) de precisão.
- Em qualquer representação em ponto flutuante temos o menor e o maior número que pode ser representado, indicando *underflow* e *overflow*, respectivamente.

Exemplo 8 Qual número é representado por 1 10000000011 10110011100010...0?

Note que o “três pontinhos” representam uma sequência de 37 zeros. O primeiro 1 se refere ao sinal do número, neste caso $-$. Depois, temos a sequência de 11 *bits* que representa o expoente: $e = 1 \times 2^{10} + 1 \times 2^1 + 1 \times 2^0 = 1027$. Em seguida, os 52 *bits* que representam a mantissa: $m = 1 \times 2^{-1} + 1 \times 2^{-3} + 1 \times 2^{-4} + 1 \times 2^{-7} + 1 \times 2^{-8} + 1 \times 2^{-9} + 1 \times 2^{-13} = 0.7012939453$. Dessa forma, temos $fl(x) = (-1)^1 2^4 (1 + m) = -27.2207031248$.

As operações em aritmética de ponto flutuante devem seguir as regras de arredondamento a cada operação. Vamos exercitar com sistemas de numeração com poucos algarismos significativos para observar o que ocorre com os cálculos feitos em computadores.

Exemplo 9 Represente os números 1.234 e 1.235 com três algarismos significativos.

Utilizando o arredondamento temos $1.234 \approx 1.23$ e $1.235 \approx 1.24$. Utilizando o corte temos $1.234 \approx 1.23$ e $1.235 \approx 1.23$.

Exemplo 10 Realize as operações usando três algarismos significativos.

(a) $(4.26 + 9.25) + 5.04$;

(b) $4.26 + (9.25 + 5.04)$.

Lembre-se ue a regra de arredondamento deve ser aplicada a cada operação. No item (a), usando o corte, temos:

$$(4.26 + 9.25) + 5.04 = 13.51 + 5.04 \approx 13.5 + 5.04 = 18.54 \approx 18.5,$$

e, note que, usando o arredondamento, temos o mesmo resultado. Já no item (b), usando o corte, temos:

$$4.26 + (9.25 + 5.04) = 4.26 + 14.29 \approx 4.26 + 14.2 = 18.46 \approx 18.4,$$

e, usando o arredondamento,

$$4.26 + (9.25 + 5.04) = 4.26 + 14.29 \approx 4.26 + 14.3 = 18.56 \approx 18.6.$$

Note que, com a aritmética de ponto flutuante perdemos a propriedade da associatividade. Precisamos estar atentos as quantidades de algarismos significativos que utilizamos nos nossos cálculos. O seguinte resultado, nos apresenta uma estimativa do erro que cometemos na representação de números reais com a aritmética de ponto flutuante.

Lema 1 *O erro na representação de um número $a \neq 0$ em aritmética de ponto flutuante com n algarismos significativos, na base decimal é limitado por,*

$$|a - fl(a)| \leq 5|a|10^{-n}p,$$

com $p = 1$ se feito o arredondamento e $p = 2$ no caso do corte.

demonstração:

Supondo $a = 10^q(0.d_1d_2 \dots d_nd_{n+1} \dots)$, usando o corte para representar a , temos:

$$fl(a) = 10^q(0.d_1d_2 \dots d_n).$$

Assim,

$$\begin{aligned} |a - fl(a)| &= 10^{q-n}(0.d_{n+1} \dots) \\ &= 10^{q-n}(d_{n+1}d_{n+2} \dots) \frac{|a|}{|a|} \\ &= 10^{-n} \frac{0.d_{n+1}d_{n+2} \dots}{0.d_1d_2 \dots} |a| \end{aligned}$$

Note que, $1 \leq d_1 \leq 9$ e $0.d_{n+1}d_{n+2} \dots \leq 1$ temos que quanto menor o denominador da expressão:

$$\frac{0.d_{n+1}d_{n+2} \dots}{0.d_1d_2 \dots} \leq \frac{1}{0.1},$$

menor o resultado.

Portanto,

$$|a - fl(a)| \leq 10^{1-n}|a|,$$

o que mostra o resultado com $p = 2$, já que utilizamos o corte.

□

Vamos encerrar a unidade apresentando o problema de perda de precisão em decorrência do erro de arredondamento, que pode ser evitada por uma reformulação do problema.

Considere o problema de determinar x tal que $ax^2 + bx + c = 0$, com $a \neq 0$. A partir de fórmula de Bhaskara as duas soluções são dadas por,

$$x_1 = \frac{-b + \sqrt{b^2 - 4ac}}{2a} \quad \text{e} \quad x_2 = \frac{-b - \sqrt{b^2 - 4ac}}{2a}.$$

Vamos aplicar a fórmula à equação $x^2 - 100.23x + 1.2371 = 0$, cujas soluções são aproximadamente, $x_{1e} = 100.21765$ e $x_{2e} = 0.012344132$, usando 5 algarismos significativos e a regra do corte para o arredondamento. Note que, nessa equação, b^2 é muito maior do que $4ac$, isso causará uma perda de precisão. Veja na sequência,

$$b^2 - 4ac = (-100.23)^2 - 4 \times 1.2371 = 10046.052 - 4.9484 \approx 10046 - 4.9484 = 10041.051$$

$$\sqrt{b^2 - 4ac} \approx \sqrt{10041} = 100.20479 \approx 100.20.$$

Assim temos,

$$x_1 = \frac{100.23 + 100.20}{2} = 100.215 \approx 100.21, \quad \text{e} \quad x_2 = \frac{100.23 - 100.20}{2} = \frac{0.03}{2} = 0.015.$$

Ao calcular o erro relativo de cada aproximação obtemos,

$$\frac{|x_{1e} - x_1|}{|x_{1e}|} \approx 7.63 \times 10^{-5} \quad \text{e} \quad \frac{|x_{2e} - x_2|}{|x_{2e}|} \approx 0.215.$$

O cálculo de x_1 envolveu a soma de números quase iguais, o que não representou nenhum problema para a aproximação. Já a aproximação de x_{2e} envolveu a subtração de números “quase iguais” (próximo de zero). Isso unido a pequena quantidade de algarismos significativos gerou uma aproximação insatisfatória, com grande erro relativo.

Note que x_1 e x_2 também podem ser determinadas por meio das regras de soma e produto,

$$x_1 + x_2 = -\frac{b}{a} \quad \text{e} \quad x_1 x_2 = \frac{c}{a}.$$

Neste caso, a aproximação para x_{2e} é dada por,

$$x_2 = \frac{1.2371}{100.21} = 0.01234507 \approx 0.012345.$$

Uma aproximação com erro relativo,

$$\frac{|x_{2e} - x_2|}{|x_{2e}|} \approx 7.03 \times 10^{-5}.$$

Assim, concluímos que é importante refletir antes de calcular.

Para finalizar e reforçar, a notação:

- Convencional: 0.3333, 1.345 e 0.002531;

- Fracionária: $\frac{1}{3}$;
- Fatorada: $1.345 = 1 \times 10^0 + 3 \times 10^{-1} + 4 \times 10^{-2} + 5 \times 10^{-3}$;
- Ponto flutuante: 0.1345×10^1 e 0.2531×10^{-2} .

Lembre-se que vamos trabalhar com a notação convencional e de ponto flutuante.

Leitura recomendada:

- Noções de Cálculo Numérico. Capítulo 1 [2].
- Cálculo Numérico Computacional. Capítulo 1 [5].

1.5 Exercícios

1. Escreva os seguintes números, na forma normalizada, na base $b = 10$ em aritmética de ponto flutuante de $n = 4$ e $n = 6$ algarismos significativos, $\pm(.d_1d_2 \dots d_n) \times b^e$. Use corte e arredondamento e compare os resultados.

(a) -77549

(c) 10.0097

(b) 0.0027823

(d) $\cos(0.7)$

2. Considere um sistema de numeração $F(2, 8, -4, 5)$.

(a) Escreva o número $(0.10110001) \times 2^3$ e o seu sucessor $(0.10110010) \times 2^3$ na base 10.

(a) A representação do número 5.5, neste sistema de numeração, é exata? Justifique.

3. Efetue as seguintes operações com aritmética de ponto flutuante, usando 3 algarismos significativos:

(a) $(2.06 - 0.0997) + (5.059 - 0.089)$
 0.7603

(b) $(1/6) \times (0.0027632) - 45.03 +$

4. Considere $x = 0.5289$, $y = 0.8012$ e $z = 0.6024$ e operações em ponto flutuante numa mantissa com 4 algarismos significativos (use o arredondamento a cada operação) e verifique se:

(a) $x \cdot (y + z) = x \cdot y + x \cdot z$

(b) $(x + y) + z = x + (y + z)$

5. Um sistema de ponto flutuante pode ser expresso pela função $F(b, n, e_1, e_2)$. Por exemplo, dado o sistema $F(10, 4, -5, 5)$, o número $x = 279.15$ é representado como $x = 0.2791 \times 10^3$, usando corte e $x = 0.2792 \times 10^3$, usando arredondamento.

Considere uma máquina com o sistema descrito acima. Pede-se:

- (a) qual o menor e o maior número (em valor absoluto) representados nesta máquina?

- (b) como será representado o número 73.758 nesta máquina, se for usado o arredondamento? E se for usado o truncamento?
 - (c) se $a = 42450$ e $b = 3$ qual o resultado de $a + b$?
 - (d) qual o resultado da soma $S = 42450 + \sum_{k=1}^{10} 3$, nesta máquina?
 - (e) o mesmo para a soma, $S = \sum_{k=1}^{10} 3 + 42450$
 - (f) o resultado da operação: wz/t pode ser obtido de várias maneiras, bastando modificar a ordem em que os cálculos são efetuados. Para determinarmos valores de w , z e t , uma sequência de cálculos pode ser melhor que outra. Faça uma análise para o caso em que $w = 100$, $z = 3500$ e $t = 7$.
6. Escreva um programa em alguma linguagem de programação para obter o resultado da seguinte operação:

$$S = 10000 - \sum_{k=1}^N x,$$

para:

- (a) $N = 10000$ e $x = 0.1$
 - (b) $N = 80000$ e $x = 0.125$
7. A série de Taylor da função $\ln(x)$ é expressa por

$$\ln(x) = \sum_{k=1}^{\infty} (-1)^{k-1} \frac{(x-1)^k}{k},$$

com raio de convergência $0 < x \leq 2$. Utilizando aritmética de ponto flutuante com 5 algarismos significativos:

- (a) Aproxime os valores de $\ln(1.5)$, $\ln(2)$ e $\ln(3)$, com $N = 10$ e 20 .
 - (b) Observe que a série para $\ln(3)$ é divergente, o cálculo pela série de Taylor fornece valores errados e, em nenhum momento vai convergir para o valor 1.0986122887 com qualquer número de termos da série. Usando a propriedade da função logarítmicas, $\ln(a \cdot b) = \ln(a) + \ln(b)$, faça $\ln(3) = \ln(2 \times 1.5)$ com $n = 20$ (já calculado no item (a)) e compare com o resultado exato.
8. No Cálculo Diferencial e Integral, demonstra-se que a série Harmônica $\sum_{k=1}^{\infty} \frac{1}{k}$ não converge. Construa um programa que efetue uma aproximação da série Harmônica e verifique este resultado computacionalmente.